WEB SERVICE VÀ CÁC KHÁI NIỆM LIÊN QUAN

Tài liệu đọc thêm — Học phần Lập trình mạng (INT1433)

1. Bắt đầu từ một tình huống

Một ngân hàng có hệ thống tính tỷ giá viết bằng Java. Một công ty du lịch muốn hiển thị tỷ giá ngay trên website viết bằng PHP; một ứng dụng di động viết bằng Kotlin cũng muốn dùng. Ngân hàng không thể đưa mã nguồn cho họ, càng không thể cho họ truy cập thẳng vào cơ sở dữ liệu. Cách giải quyết: ngân hàng **mở một "cửa" trên Internet** — gửi tới cửa đó câu hỏi "100 USD bằng bao nhiêu VND?" theo một khuôn dạng thống nhất, cửa sẽ trả lời theo cùng khuôn dạng đó. Cái "cửa" ấy chính là **web service**.

So sánh để dễ nhớ: một **ứng dụng web** dùng HTTP để phục vụ **con người** (trả về trang HTML cho trình duyệt hiển thị); một **web service** cũng dùng HTTP nhưng phục vụ **chương trình khác** (trả về dữ liệu có cấu trúc để chương trình đọc). Có thể hình dung web service như **quầy giao dịch của ngân hàng**: khách không cần biết bên trong ngân hàng làm việc thế nào, chỉ cần điền đúng mẫu đơn và nộp đúng quầy.

2. Web service là gì

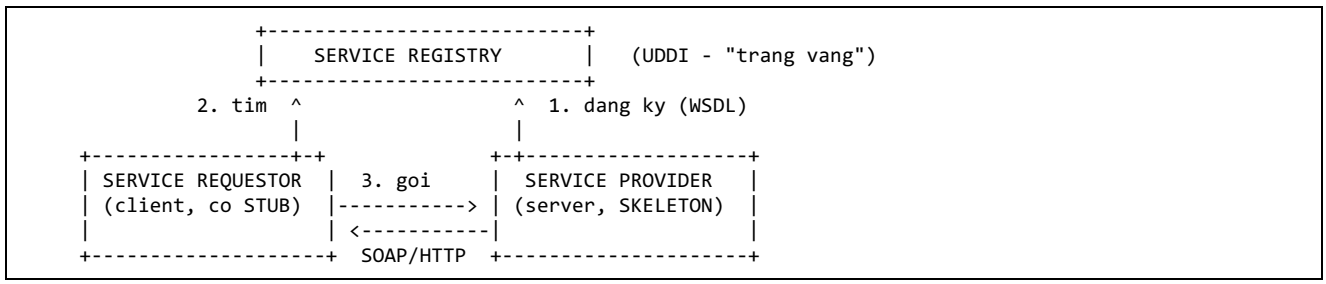
Theo W3C, web service là **một hệ thống phần mềm được định danh bằng URI, có giao tiếp công khai được mô tả bằng XML; các hệ thống khác có thể tìm thấy mô tả đó và tương tác với dịch vụ bằng các thông điệp XML truyền qua giao thức Internet.** Từ định nghĩa này rút ra các đặc điểm chính:

Đặc điểm	Ý nghĩa	Ví dụ
Định danh bằng URI	Mỗi dịch vụ có một địa chỉ như trang web	http://bank.vn/ws/tygia
Giao tiếp được mô tả công khai	Có "bản hướng dẫn sử dụng" máy đọc được: có những hàm nào, tham số gì	Tài liệu WSDL
Trao đổi bằng thông điệp chuẩn	Dữ liệu là XML (hoặc JSON với REST) — văn bản mọi ngôn ngữ đều đọc được	Thông điệp SOAP
Truyền qua giao thức Internet	Thường là HTTP, cổng 80/443, đi qua tường lửa như trang web	HTTP POST
Liên kết lỏng (loosely coupled)	Client và server độc lập: đổi ngôn ngữ, đổi máy chủ bên này không ảnh hưởng bên kia, miễn giữ đúng giao tiếp	Server Java, client PHP/C#/Python

Nhờ vậy web service **không phụ thuộc ngôn ngữ lập trình và hệ điều hành**, dùng lại được, và là nền tảng của **kiến trúc hướng dịch vụ (SOA)**: một hệ thống lớn được ghép từ nhiều dịch vụ nhỏ gọi nhau qua mạng. Ứng dụng điển hình: tích hợp giữa doanh nghiệp (B2B), thanh toán trực tuyến, tra cứu thông tin (tỷ giá, thời tiết, vận đơn), kết nối hệ thống nội bộ của các phòng ban.

3. Kiến trúc: ba vai trò và ba công nghệ

Mô hình web service cổ điển có **ba vai trò** và **ba công nghệ**, tất cả đều dựa trên XML:



Công nghệ	Vai trò	Hình dung
SOAP	Giao thức đóng gói thông điệp XML để gửi đi (wire stack)	Phong bì thư theo mẫu chuẩn
WSDL	Ngôn ngữ mô tả dịch vụ: hàm, tham số, kiểu, địa chỉ (description stack)	Bản hướng dẫn sử dụng / hợp đồng
UDDI (và WSIL)	Danh bạ để công bố và tìm dịch vụ (discovery stack)	Trang vàng danh bạ doanh nghiệp

Quy trình: (1) nhà cung cấp viết dịch vụ, mô tả nó bằng WSDL và đăng ký vào danh bạ; (2) người dùng tìm trong danh bạ và lấy WSDL; (3) từ WSDL, công cụ sinh ra **stub** — một lớp đại diện ở phía client; chương trình gọi hàm của stub như gọi hàm thường, stub đóng gói lời gọi thành SOAP và gửi đi; phía server, **skeleton** mở gói, gọi hàm thật và gửi kết quả về. Cơ chế stub/skeleton này giống hệt RMI, chỉ khác là dữ liệu đi bằng XML qua HTTP nên mọi ngôn ngữ đều tham gia được. Trong thực tế UDDI công cộng không thành công (đóng cửa năm 2006); WSDL thường được lấy trực tiếp tại địa chỉ dịch vụ, ví dụ `http://bank.vn/ws/tygia?wsdl`.

4. SOAP — phong bì của thông điệp

SOAP (Simple Object Access Protocol) quy định cách bọc dữ liệu XML thành một thông điệp. Giống một lá thư, thông điệp SOAP gồm **phong bì** `Envelope`, bên trong có phần **đầu** `Header` (tùy chọn — thông tin phụ như mã phiên, xác thực) và phần **thân** `Body` (bắt buộc — nội dung chính). Client gửi thông điệp bằng **HTTP POST**, server trả lời bằng một thông điệp SOAP khác. Lời gọi `getRate("USD", "VND")` được gửi đi như sau:

```
POST /ws/tygia HTTP/1.1
Content-Type: text/xml; charset=utf-8
SOAPAction: "urn:tygia#getRate"


<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Header>
    <auth:Token xmlns:auth="urn:auth">A1B2C3</auth:Token>   <!-- tuy chon -->
  </soap:Header>
  <soap:Body>
    <getRate xmlns="urn:tygia">          <!-- ten ham -->
      <from>USD</from>                     <!-- tham so 1 -->
      <to>VND</to>                         <!-- tham so 2 -->
    </getRate>
  </soap:Body>
</soap:Envelope>
```

Server trả lời bằng `<getRateResponse><return>25000</return></getRateResponse>` trong Body. Khi có lỗi (tên hàm sai, tham số không hợp lệ...), Body chứa phần tử `**<soap:Fault>**` gồm `faultcode` (lỗi do client hay server) và `faultstring` (mô tả lỗi) — tương tự ngoại lệ trong Java. Quy tắc dễ nhớ: **tên hàm là phần tử cấp 1 trong Body, tên tham số là phần tử cấp 2, giá trị là nội dung**.

5. WSDL — bản hợp đồng của dịch vụ

Làm sao client biết dịch vụ có hàm `getRate`, nhận hai chuỗi và trả về một số? Câu trả lời là **WSDL** (Web Services Description Language) — một tài liệu XML mô tả đầy đủ dịch vụ, đóng vai trò **hợp đồng** giữa hai bên. Có thể so sánh WSDL với một **interface trong Java** cộng thêm địa chỉ và cách liên lạc. WSDL 1.1 gồm 5 phần chính, đọc từ "trừu tượng" tới "cụ thể":

Phần tử	Trả lời câu hỏi	Tương đương trong Java
types	Dùng những kiểu dữ liệu nào? (định nghĩa bằng XML Schema)	Các lớp dữ liệu, kiểu tham số
message	Mỗi lời gọi gửi đi / nhận về những dữ liệu gì?	Danh sách tham số, kiểu trả về
portType	Dịch vụ có những thao tác (operation) nào?	interface với các phương thức
binding	Gửi thông điệp bằng giao thức gì, định dạng nào? (SOAP/HTTP, rpc/document)	Cách triển khai giao thức
service / port	Dịch vụ nằm ở địa chỉ nào?	URL của máy chủ

WSDL rút gọn cho dịch vụ tỷ giá ở trên:

```
<definitions name="TyGiaService" targetNamespace="urn:tygia" xmlns:tns="urn:tygia"
    xmlns="http://schemas.xmlsoap.org/wsdl/" xmlns:xsd="http://www.w3.org/2001/XMLSchema"
    xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/">
  <message name="getRateRequest">                        <!-- tham so vao -->
    <part name="from" type="xsd:string"/>
    <part name="to"   type="xsd:string"/>
  </message>
  <message name="getRateResponse">                       <!-- ket qua -->
    <part name="return" type="xsd:double"/>
  </message>
  <portType name="TyGiaPort">                            <!-- "interface" -->
    <operation name="getRate">
      <input  message="tns:getRateRequest"/>
      <output message="tns:getRateResponse"/>
    </operation>
  </portType>
  <binding name="TyGiaBinding" type="tns:TyGiaPort">     <!-- SOAP qua HTTP -->
    <soap:binding style="rpc" transport="http://schemas.xmlsoap.org/soap/http"/>
    <operation name="getRate"> ... </operation>
  </binding>
  <service name="TyGiaService">                          <!-- dia chi -->
    <port name="TyGiaPort" binding="tns:TyGiaBinding">
      <soap:address location="http://bank.vn/ws/tygia"/>
    </port>
  </service>
</definitions>
```

Đọc từ trên xuống: *dịch vụ TyGiaService có thao tác getRate, nhận hai chuỗi from, to, trả về một số thực; thông điệp được gửi bằng SOAP qua HTTP tới http://bank.vn/ws/tygia.* Từ tài liệu này, các công cụ như `wsimport` (Java), Visual Studio (.NET) tự sinh ra stub, ví dụ giao tiếp Java `interface TyGiaPort { double getRate(String from, String to); }` — lập trình

viên phía client không phải viết một dòng XML nào. **targetNamespace** (`urn:tygia`) là "họ" của các tên trong WSDL, giúp phân biệt với tên trùng của dịch vụ khác; tiền tố `tns:` trỏ về chính namespace đó, `xsd:` trỏ về XML Schema.

6. XML Schema — thống nhất kiểu dữ liệu

Hai chương trình viết bằng hai ngôn ngữ cần thống nhất "thế nào là một số thực, một ngày tháng". **XML Schema** (tiền tố `xsd:`) cung cấp các kiểu có sẵn — `xsd:string`, `xsd:int`, `xsd:double`, `xsd:boolean`, `xsd:decimal`, `xsd:date` (dạng 2026-11-03)... — và cho phép tự định nghĩa kiểu mới trong phần `types` của WSDL:

- **simpleType** — hạn chế một kiểu có sẵn bằng các ràng buộc (facet): `enumeration` (chỉ được chọn trong danh sách), `minInclusive`/`maxInclusive` (khoảng giá trị), `length`, `pattern` (biểu thức chính quy)... Ví dụ: mã tiền tệ là chuỗi 3 chữ in hoa `[A-Z]{3}`.
- **complexType** — kiểu có cấu trúc, gồm nhiều phần tử con (`<xsd:sequence>`), tương đương một lớp Java có nhiều thuộc tính. Ví dụ: kiểu `ExchangeValue` gồm `currency`, `value`, `date`.

```
<xsd:simpleType name="CurrencyCode">           <!-- chuoi 3 chu in hoa -->
  <xsd:restriction base="xsd:string">
    <xsd:pattern value="[A-Z]{3}"/>
  </xsd:restriction>
</xsd:simpleType>
<xsd:complexType name="ExchangeValue">         <!-- giong mot lop Java -->
  <xsd:sequence>
    <xsd:element name="currency" type="tns:CurrencyCode"/>
    <xsd:element name="value"    type="xsd:double"/>
    <xsd:element name="date"     type="xsd:date"/>
  </xsd:sequence>
</xsd:complexType>
```

Công cụ sinh code ánh xạ các kiểu này sang ngôn ngữ đích: `xsd:int` → `int`, `xsd:string` → `String`, `xsd:decimal` → `BigDecimal`, `xsd:date` → `XMLGregorianCalendar` (JAX-WS), complexType → một lớp Java có các thuộc tính tương ứng.

7. Web service trong Java và hiện nay

Công nghệ	Thời điểm	Ghi chú
JAX-RPC	J2EE 1.4 (2003)	API đầu tiên cho web service SOAP; đã lỗi thời.
JAX-WS	Java EE 5 (2006); có sẵn trong JDK 6–8	Viết dịch vụ bằng annotation `@WebService`, `@WebMethod`; tự sinh WSDL; `wsimport` sinh stub. Từ JDK 11 phải thêm thư viện (Jakarta XML Web Services).
JAX-RS	Java EE 6 (2009)	Dành cho web service kiểu **REST** (không dùng SOAP/WSDL).

Dịch vụ JAX-WS tối thiểu chỉ cần vài dòng; WSDL được sinh tự động tại `http://localhost:8080/tygia?wsdl`:

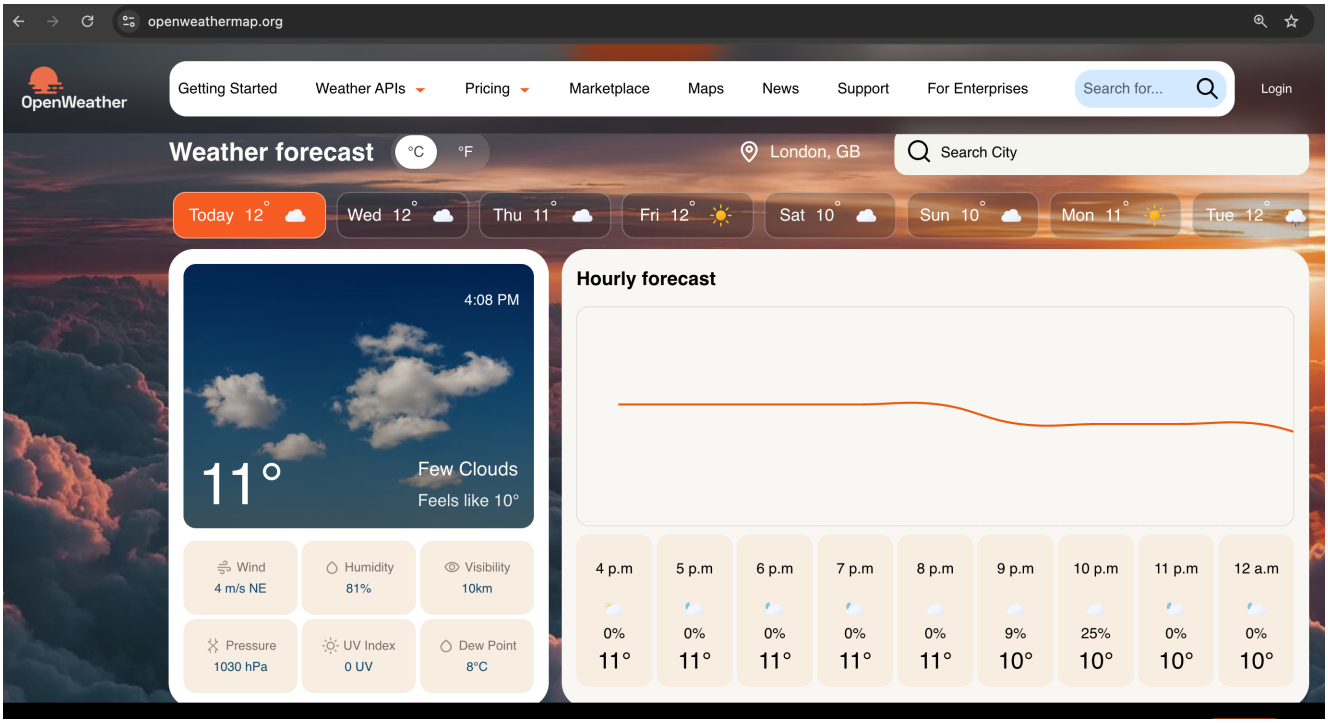
```
@WebService
public class TyGia {
    @WebMethod
    public double getRate(String from, String to) { return 25000; }

    public static void main(String[] args) {
        Endpoint.publish("http://localhost:8080/tygia", new TyGia());
    }
}
```

SOAP và REST. Ngày nay phần lớn dịch vụ mới dùng phong cách **REST**: không có phong bì SOAP, mỗi đối tượng là một **tài nguyên** có URL riêng (`/api/tygia/USD`), thao tác bằng chính các phương thức HTTP (GET, POST, PUT, DELETE), dữ liệu thường là **JSON** gọn nhẹ (`{"from":"USD","rate":25000}`), và tài liệu mô tả là **OpenAPI** thay cho WSDL. SOAP/WSDL vẫn được dùng nhiều trong ngân hàng, bảo hiểm, cơ quan nhà nước — nơi cần hợp đồng chặt chẽ, kiểm tra kiểu và các chuẩn bảo mật thông điệp (WS-Security).

	SOAP + WSDL	**REST + JSON**
Thông điệp	XML trong phong bì SOAP, khá dài	JSON gọn, hoặc XML
Giao thức	Thường HTTP POST cho mọi thao tác	Dùng đúng GET/POST/PUT/DELETE và mã trạng thái HTTP
Mô tả dịch vụ	WSDL (bắt buộc với công cụ sinh stub)	OpenAPI (tùy chọn)
Thế mạnh	Hợp đồng chặt, kiểu dữ liệu rõ, chuẩn bảo mật	Đơn giản, nhẹ, dễ gọi từ web và di động

8. Ví dụ thực tế: web service thời tiết OpenWeatherMap



OpenWeatherMap (openweathermap.org) cung cấp dữ liệu thời tiết hiện tại, dự báo và lịch sử cho mọi địa điểm trên thế giới qua web service. Hàng nghìn website và ứng dụng di động hiển thị thời tiết bằng cách gọi dịch vụ này thay vì tự đặt trạm đo. Đây là một web service kiểu **REST**: không có phong bì SOAP hay WSDL, mỗi lời gọi chỉ là một **HTTP GET** tới một URL, kết quả trả về là **JSON** (hoặc XML nếu yêu cầu).

Chuẩn bị: đăng ký tài khoản miễn phí, vào mục *API keys* để lấy **API key** — một chuỗi bí mật gửi kèm mỗi lời gọi qua tham số `appid` để dịch vụ biết ai đang gọi. Key mới được kích hoạt tự động trong khoảng 2 giờ. Gói miễn phí cho phép 60 lời gọi/phút và 1.000.000 lời gọi/tháng (theo trang giá của OpenWeatherMap tại thời điểm biên soạn).

Cấu trúc một lời gọi — API thời tiết hiện tại (Current Weather API):

```
https://api.openweathermap.org/data/2.5/weather?q=Hanoi&units=metric&lang=vi&appid={API_KEY}
\_____________________________/\_____________/\_____________________________________/
        dia chi may chu           tai nguyen                tham so (query string)
```

Tham số	Bắt buộc	Ý nghĩa
`appid`	Có	API key của người gọi.
`lat`, `lon`	Một trong hai cách	Vĩ độ, kinh độ — cách chính xác nhất, được khuyến nghị (ví dụ lat=21.0245&lon=105.8412).
`q`	Một trong hai cách	Tên thành phố, có thể kèm mã quốc gia (q=Hanoi,VN). Tài liệu ghi cách này đã deprecated (vẫn hoạt động), khuyên dùng Geocoding API để đổi tên thành tọa độ.
`units`	Không	`standard` (mặc định, độ Kelvin), `metric` (độ C), `imperial` (độ F).
`lang`	Không	Ngôn ngữ của phần mô tả, ví dụ `vi` cho tiếng Việt.
`mode`	Không	Định dạng kết quả: JSON (mặc định), `xml` hoặc `html`.

Kết quả trả về (JSON, rút gọn; giá trị số liệu chỉ để minh họa — cấu trúc theo tài liệu của OpenWeatherMap):

```
{
  "coord":   { "lon": 105.8412, "lat": 21.0245 },
  "weather": [ { "id": 803, "main": "Clouds", "description": "mây cụm", "icon": "04n" } ],
  "main":    { "temp": 24.5, "feels_like": 25.1, "pressure": 1014, "humidity": 78 },
  "visibility": 10000,
  "wind":    { "speed": 2.1, "deg": 60 },
  "clouds":  { "all": 75 },
  "dt": 1793718000,
  "sys":     { "country": "VN", "sunrise": 1793660400, "sunset": 1793701500 },
  "timezone": 25200,
  "name": "Hà Nội",
  "cod": 200
}
```

Đọc kết quả: `main.temp` là nhiệt độ (độ C vì đã đặt units=metric), `main.humidity` độ ẩm (%), `weather[0].description` mô tả bằng tiếng Việt, `wind.speed` tốc độ gió (m/s), `dt`, `sunrise`, `sunset` là thời điểm dạng Unix (số giây từ 1/1/1970, giờ UTC), `timezone` là độ lệch múi giờ tính bằng giây (25200 giây = UTC+7). Thêm `&mode=xml` thì cùng dữ liệu được trả về dưới dạng XML — cho thấy một dịch vụ có thể cung cấp nhiều **biểu diễn** cho cùng một tài nguyên.

Khi có lỗi, dịch vụ trả về **mã trạng thái HTTP** kèm một JSON ngắn có `cod` và `message` (khác SOAP, nơi lỗi nằm trong `<soap:Fault>`):

Mã HTTP	Nguyên nhân	Ví dụ nội dung trả về
401 Unauthorized	Thiếu API key, key sai hoặc chưa được kích hoạt	{"cod":401, "message": "Invalid API key. ..."}
404 Not Found	Tên thành phố / mã sai, hoặc sai định dạng yêu cầu	{"cod":"404", "message":"city not found"}
429 Too Many Requests	Vượt giới hạn số lời gọi của gói	—

Gọi từ trình duyệt hoặc dòng lệnh: dán URL ở trên (thay {API_KEY} bằng key của mình) vào thanh địa chỉ trình duyệt, hoặc dùng lệnh curl với cùng URL. **Gọi từ Java** bằng `java.net.http.HttpClient` (JDK 11+):

```
public class WeatherClient {
    public static void main(String[] args) throws Exception {
        String key = args[0];                                   // API key
        String city = args.length > 1 ? args[1] : "Hanoi";
        String url = "https://api.openweathermap.org/data/2.5/weather"
                + "?q=" + URLEncoder.encode(city, StandardCharsets.UTF_8)
                + "&units=metric&lang=vi&appid=" + key;


        HttpResponse<String> r = HttpClient.newHttpClient().send(
                HttpRequest.newBuilder(URI.create(url)).GET().build(),
                HttpResponse.BodyHandlers.ofString());


        if (r.statusCode() != 200) {                            // 401, 404, 429...
            System.out.println("Loi HTTP " + r.statusCode() + ": " + lay(r.body(), "message"));
            return;
        }
        String json = r.body();
        System.out.println("Thanh pho : " + lay(json, "name"));
        System.out.println("Thoi tiet : " + lay(json, "description"));
        System.out.println("Nhiet do  : " + lay(json, "temp") + " do C");
        System.out.println("Do am     : " + lay(json, "humidity") + " %");
    }


    /** Lay gia tri cua mot khoa trong JSON (cach don gian; thuc te dung Gson/Jackson). */
    static String lay(String json, String khoa) {
        Matcher m = Pattern.compile("\"" + khoa + "\"\\s*:\\s*\"?([^\",}]*)").matcher(json);
        return m.find() ? m.group(1) : "?";
    }
}
```

Kết quả chạy với dữ liệu mẫu ở trên, với key sai, và với tên thành phố không tồn tại:

```
$ java WeatherClient <API_KEY> Hanoi
Thanh pho : Hà Nội
Thoi tiet : mây cụm
Nhiet do  : 24.5 do C
Do am     : 78 %
$ java WeatherClient SAI_KEY Hanoi
Loi HTTP 401: Invalid API key. Please see https://openweathermap.org/faq#error401 for more info.
$ java WeatherClient <API_KEY> Atlantis
Loi HTTP 404: city not found
```

Đối chiếu với các khái niệm đã học — OpenWeatherMap vẫn có đủ các thành phần của một web service, chỉ là ở dạng REST:

Khái niệm	Web service SOAP (mục 3–6)	OpenWeatherMap (REST)
Địa chỉ dịch vụ (URI)	soap:address trong WSDL	https://api.openweathermap.org/data/2.5/weather
Mô tả dịch vụ	Tài liệu WSDL máy đọc được	Trang tài liệu API (openweathermap.org/current)
Gửi lời gọi và tham số	Phong bì SOAP, HTTP POST	HTTP GET, tham số trên query string
Dữ liệu trả về, kiểu dữ liệu	XML, kiểu định nghĩa bằng XML Schema	JSON (hoặc XML với mode=xml), kiểu mô tả trong tài liệu
Báo lỗi	<soap:Fault>	Mã trạng thái HTTP + JSON {cod, message}
Xác thực	Header SOAP (WS-Security)	API key trong tham số appid
Stub phía client	Sinh tự động từ WSDL (wsimport)	Tự viết bằng HttpClient, hoặc dùng thư viện do cộng đồng viết

9. Tóm tắt

- **Web service** là chức năng được "mở" qua mạng cho chương trình khác gọi, độc lập ngôn ngữ và nền tảng.
- **SOAP** = phong bì XML (Envelope, Header, Body, Fault) gửi bằng HTTP POST.
- **WSDL** = hợp đồng mô tả dịch vụ: types → message → portType → binding → service; từ WSDL công cụ sinh ra stub.
- **XML Schema** = thống nhất kiểu dữ liệu (simpleType, complexType) giữa các ngôn ngữ.
- **UDDI/WSIL** = công bố và tìm dịch vụ; ít dùng trong thực tế.
- Java: JAX-WS cho SOAP, JAX-RS cho REST; dịch vụ mới phổ biến nhất hiện nay là **REST + JSON**, ví dụ OpenWeatherMap: gọi bằng HTTP GET với API key, nhận JSON, lỗi báo bằng mã trạng thái HTTP.